

Leakage-Aware Privacy-Practice Classification: A Comparative Study of Classical and Neural Models under Policy-Disjoint Evaluation

Nafiz Ahmed Nafi¹ Priom Halder¹ MD. Amirul Islam Sadat¹ Samiha Tasnim Orthi¹

¹Department of Computer Science and Engineering, BRAC University

nafiz.ahmed.nafi@g.bracu.ac.bd priom.halder@g.bracu.ac.bd

amirul.islam.sadat@g.bracu.ac.bd samiha.tasnim.orthi@g.bracu.ac.bd

August 28, 2026

Abstract

Privacy policies govern how personal data is collected, shared, and retained, yet most users never read them in full. We study automatic classification of annotated privacy-policy spans into eight data-practice categories using the OPP-115 corpus (17,343 labeled spans, 115 policies). Representations include TF-IDF and a partially trained word2vec embedding; models span three classical approaches (Random Forest, Logistic Regression, Naive Bayes) and seven neural architectures (SimpleRNN, GRU, LSTM, their bidirectional variants, and a partially fine-tuned BERT Base). Our central contribution is a systematic comparison of two evaluation protocols: a conventional row-level stratified split and a policy-disjoint split that holds entire policies out of training. Because privacy policies reuse boilerplate language across unrelated documents, the row-level split inflates macro-F1 by 0.015–0.123 depending on the model and changes the apparent ranking. Under the honest, policy-disjoint evaluation, BERT Base achieves the highest macro-F1 (0.706), narrowly ahead of Naive Bayes (0.701), while showing the smallest evaluation-protocol gap of any model trained. The results suggest that document-grouped evaluation should be standard practice for any text classification task built on corpora with reused or template-based language.

1 Introduction

Privacy policies are the main legal instrument through which organizations disclose how they collect, use, share, and retain personal data. In practice they are long, formulaically written, and inconsistently structured, and surveys of user behavior repeatedly find that almost nobody reads one in full before consenting (McDonald & Cranor, 2008). Automatically mapping policy text onto a standardized taxonomy of data practices would let downstream tools summarize a policy at a glance, flag GDPR or CCPA-relevant language, or let researchers compare practices at corpus scale. This is a multi-class text classification problem and not a saturated task: privacy language is technical, class frequencies are heavily skewed, and a single clause often expresses more than one practice at once.

We build our study on the OPP-115 corpus (Wilson et al., 2016), 115 real-world website privacy policies annotated by legal experts against a fixed schema of data-practice categories. We treat each annotated span as one classification instance labeled with one of eight practice categories.

A less obvious property of this corpus motivates the paper’s central contribution. Privacy policies reuse legally vetted boilerplate language across unrelated organizations – template clauses drafted by the same law firm recur verbatim across dozens of policies. A conventional row-level train/test split ignores this structure and can place near-identical boilerplate on both sides, letting a model score well by recognizing memorized phrases rather than by learning what privacy language of a given category actually looks like. This is an instance of the broader evaluation-leakage problem documented in NLP by Gorman & Bedrick (2019) and Lewis et al. (2021).

Research questions: *RQ1*. How does a conventional row-level split change measured performance relative to a policy-disjoint split for the same model? *RQ2*. How does this evaluation-protocol gap vary across classical, recurrent, and transformer architectures? *RQ3*. Which model achieves the best macro-F1 on entirely unseen policies?

Contributions: (1) A leakage-aware evaluation framework contrasting row-level and policy-disjoint splits across ten models spanning four architecture families. (2) An empirical finding that the evaluation-protocol gap varies substantially across model families. (3) A full implementation of the required classical and neural suite, with implementation gaps (Section 4.4) documented openly rather than omitted. Context-aware modeling, imbalance-aware training, and external validation are left for future work (Section 5).

2 Related Work

Privacy-policy NLP. The OPP-115 corpus (Wilson et al., 2016) is the standard manually annotated benchmark for data-practice classification in privacy policies. Harkous et al. (2018)

present Polisis, a deep-learning pipeline that classifies policy segments and supports downstream question answering, establishing that neural models can substantially outperform earlier rule-based approaches. Our work differs in focus: rather than proposing a new architecture, we study how the evaluation protocol – specifically, whether the split respects document-level structure – changes which architecture appears to win.

Classification models. TF-IDF paired with classical classifiers remains a strong, fast baseline (Pedregosa et al., 2011). Word embeddings such as word2vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014) enable generalization across related terms. Recurrent architectures – LSTM (Hochreiter & Schmidhuber, 1997), GRU (Cho et al., 2014), and their bidirectional variants (Schuster & Paliwal, 1997) – model sequential dependencies that bag-of-words representations discard. BERT (Devlin et al., 2019) adds contextual representations from large external corpora before task-specific fine-tuning.

Evaluation leakage and class imbalance. Gorman & Bedrick (2019) show that re-splitting standard NLP benchmarks changes both absolute scores and relative rankings. Lewis et al. (2021) document a related failure in question answering where train/test overlap inflates reported accuracy. Our work applies the same principle – splits must respect data grouping structure – to a domain where that structure is unusually explicit. Johnson & Khoshgoftaar (2019) survey class-imbalance mitigations; we adopt macro-F1 as the primary metric and include class-weighted configurations throughout.

3 Methodology

3.1 Dataset and Exploratory Data Analysis

Corpus. We use the OPP-115 corpus (Wilson et al., 2016): 115 website privacy policies, each annotated by three annotators against a fixed schema, later merged at a 0.75 overlap-similarity threshold that reproduces the corpus’s own 92/23 policy train/test split. We take the longest populated selected span as the classification input and attach its surrounding clause as optional context. Two of the ten original categories, *Other* and *Do Not Track*, are dropped, leaving eight: *First Party Collection/Use*, *Third Party Sharing/Collection*, *User Choice/Control*, *Data Security*, *User Access, Edit and Deletion*, *International and Specific Audiences*, *Policy Change*, and *Data Retention*.

Scale. The working dataset contains 17,343 labeled spans from 3,204 distinct underlying clauses and 115 policies. Of the 17,343 rows, 4,959 are repeated (span, category) pairs, an early signal of the boilerplate reuse that motivates our evaluation design. Annotated spans are short (mean 137 characters; median 15 words); surrounding clauses are longer (median 89 words).

Class distribution and span lengths. Figure 1 shows the class distribution and span/clause-length distributions. *First Party Collection/Use* and *Third Party Sharing/Collection* together account for 74.8% of all rows, while *Data Retention*

accounts for only 1.9% – a $24.3\times$ imbalance. This is why accuracy is never reported as a standalone metric anywhere in this paper; every evaluation centers macro-F1 and reports full per-class breakdowns. Span lengths are mostly short, with a long right tail (maximum 1,873 characters); this motivates the 48-token sequence-length cap used for the recurrent models.

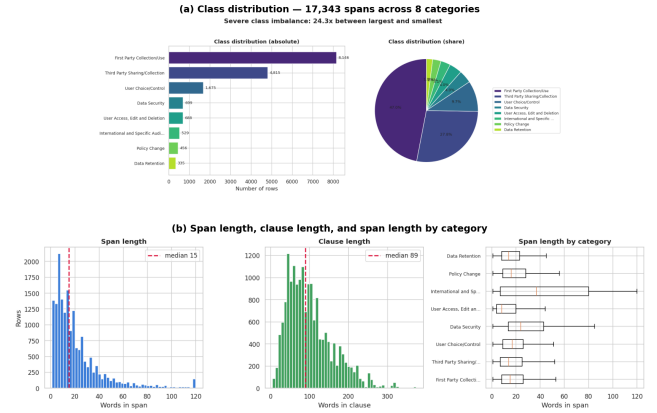


Figure 1: (a) Category distribution over 17,343 spans ($24.3\times$ largest/smallest ratio). (b) Span length, clause length, and span length by category; dashed lines mark the median.

Multi-practice clauses. 1,119 of 3,204 distinct clauses (34.9%) carry more than one category simultaneously – a sentence such as “we collect your email address and may share it with advertising partners unless you opt out” simultaneously expresses *First Party Collection/Use*, *Third Party Sharing/Collection*, and *User Choice/Control*. This is why the clause is never used as the sole classification input; a clause-level single-label formulation would force one label onto text that legitimately expresses several practices.

Boilerplate reuse. Under the conventional row-level random split, 113 of 115 policies appear on both sides and 95.2% of test rows share clause text with some training row. Under the policy-disjoint split, that figure drops to 6.0% – real-world boilerplate reuse across unrelated organizations rather than an artifact of the split. Figure 2 visualizes this contrast. The 95.2%-versus-6.0% difference is the concrete motivation for the dual-scheme evaluation in Section 3.5.

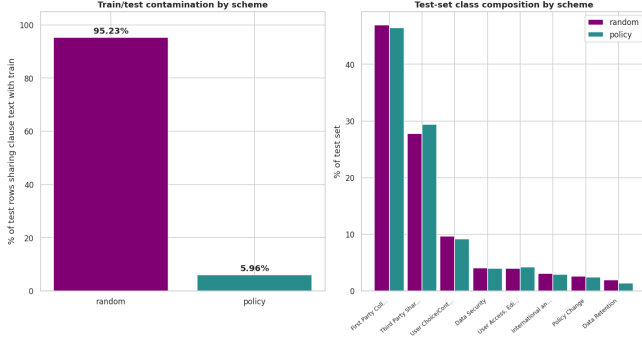


Figure 2: Left: percentage of test rows sharing clause text with training, by split scheme. Right: test-set class composition under each scheme. The two schemes differ not only in overlap but also in which policies compose the test set, so the macro-F1 gap between them is an evaluation-protocol gap, not a pure causal measure of leakage.

3.2 Preprocessing

We define five preprocessing variants, each matched to a different downstream representation. Table 1 summarises them. Every variant first strips HTML markup and replaces URLs and email addresses with placeholder tokens ([URL], [EMAIL]). In the executed pipeline, the *base* variant was applied to both the classical and recurrent models; *minimal* was applied to BERT Base. The designed-but-unexecuted match between *lemmatized* and the recurrent models is reported as a limitation in Section 5.

Table 1: Preprocessing variants and their intended usage.

Variant	Processing	Intended for
base	HTML removal, placeholders, lowercase	Classical, RNN (actual)
no_stopwords	base + NLTK stopwords removal	Alt. TF-IDF
lemmatized	base + WordNet lemmatization	RNN (designed)
aggressive	stopwords + lemmatization	Sparse configs
minimal	HTML removal, lowercase only	BERT Base

Figure 3 shows vocabulary size and mean span length for each variant on a 3,000-span sample. Stopword removal and lemmatization each reduce vocabulary size by roughly a third relative to *base*; combining both (*aggressive*) gives the smallest vocabulary. *lemmatized* has a higher mean word count than *base* despite its smaller vocabulary, most likely because the lemmatizer splits some contracted forms that the *base* tokenizer keeps intact.

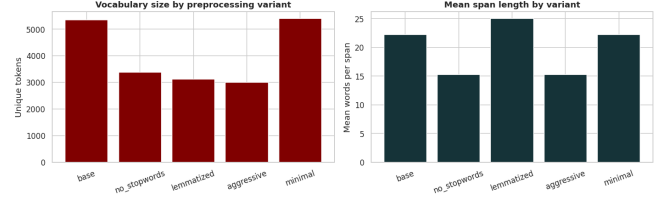


Figure 3: Vocabulary size and mean span length for each preprocessing variant, measured on a 3,000-span sample. Stopword removal and lemmatization each reduce vocabulary by roughly a third relative to *base*.

3.3 Word Representations

TF-IDF (fully evaluated): 5,000-term vocabulary, unigrams and bigrams, minimum document frequency 5, maximum document frequency 0.8, sublinear term-frequency scaling (Pedregosa et al., 2011). The vectorizer is refit independently on the training portion of each split scheme to prevent vocabulary leakage.

word2vec (partially implemented): skip-gram model (Mikolov et al., 2013) via gensim (Řehůřek & Sojka, 2010), 100 dimensions, window 5, minimum token count 2, mean document vectors. The model trains successfully on the training-split vocabulary, but no downstream classifier was fit on the resulting vectors in this project; no word2vec classification result appears in Section 4.

GloVe (Pennington et al., 2014): not implemented in the executed notebook.

3.4 Model Architectures and Hyperparameter Grids

Table 2 summarises all ten models, their representations, and the configurations tested. For every model and split scheme, model selection uses validation macro-F1 and only the winning configuration is scored on the held-out test set, so the number of configurations tried never inflates the reported test score.

Table 2: Model architectures and hyperparameter configurations (3 configs per model $\times$ 2 schemes = 60 tuning runs total).

Model	Architecture / repr.	Configurations tested
Random Forest	TF-IDF	200 trees/unlim. depth; 400/depth 60; 300/depth 30, class-weighted
Logistic Reg.	TF-IDF	$C=1.0$ (1k iter); $C=5.0$ (2k iter); $C=1.0$, class-weighted
Naive Bayes	TF-IDF	$\alpha=1.0; 0.3; 0.05$
SimpleRNN / Bi	Embed $\rightarrow$ RNN $\rightarrow$ dropout $\rightarrow$ softmax	embed 64/128; hidden 32/64; dropout 0.3/0.5; lr $10^{-3}/5 \times 10^{-4}$; one class-weighted
GRU / BiGRU	Embed $\rightarrow$ GRU $\rightarrow$ dropout $\rightarrow$ softmax	same grid
LSTM / BiLSTM	Embed $\rightarrow$ LSTM $\rightarrow$ dropout $\rightarrow$ softmax	same grid
BERT Base	Last 2 encoder layers + classifier (13.5% params trainable)	lr $\in \{2 \times 10^{-5}, 3 \times 10^{-5}, 5 \times 10^{-5}\}$; batch $\in \{16, 32\}$; one class-weighted

All six recurrent models share the same design: an embedding layer initialized from scratch, a recurrent or bidirectional recurrent layer, dropout regularization, and a dense softmax head. Input sequences are word-index-encoded from a training-only vocabulary and padded or truncated to 48 tokens. Sequence lengths are not tracked separately; the recurrent layer reads through trailing padding positions rather than stopping at the actual span end (lack of length-aware packing is noted under Limitations). Models are implemented in PyTorch (Paszke et al., 2019); final experiments ran in a CUDA-enabled Python 3.13 environment. BERT Base is fine-tuned via the HuggingFace Transformers library (Wolf et al., 2020) with the token embeddings and first 10 of 12 encoder layers frozen, and input truncated to 64 tokens.

3.5 Split Schemes

random: Rows are split 70/15/15 (train/val/test), stratified by category, with no regard for policy origin (12,140/2,601/2,602 rows; 113 of 115 policies appear in both train and test). Reported here only as a point of comparison against the honest condition; never used as a standalone generalization estimate.

policy: Policies, not rows, are the unit of assignment, built from the corpus’s own 92/23 policy split (76/16/23 policies; 11,344/2,088/3,911 rows). No policy’s rows appear in more than one split. This is the paper’s generalization estimate and the scheme used for every model-selection decision outside the explicit leakage comparison in Section 4.2.

4 Results

All ten models were trained and evaluated under both split schemes for a combined 60 logged tuning runs (18 classical, 36 recurrent, 6 BERT Base). All numbers are drawn directly from the executed project notebook (`project_440.ipynb`).

4.1 Model Comparison

Table 3 reports test-set performance and Figure 4 visualizes the comparison.

Under the honest, policy-disjoint scheme, BERT Base reaches the best overall macro-F1 (0.706), narrowly ahead of Naive Bayes (0.701) – a gap of 0.005 that should be read as a near-tie rather than a decisive win. Logistic Regression comes third (0.692). SimpleRNN is the worst model overall (macro-F1 0.131, barely above a majority-class-only baseline of ≈ 0.08), and Random Forest is the weakest of the three classical models (0.652).

Under the leaky random scheme the picture looks quite different: Logistic Regression appears to lead (0.753) and BERT falls to fourth, behind both Random Forest and Naive Bayes. A concrete instance of the accuracy trap appears within the classical models: Random Forest has the highest accuracy of all ten models under the leaky split (0.780) yet the lowest macro-F1 among the three classical models (0.721), a pattern consistent with the model benefiting disproportionately from

the majority class’s repeated boilerplate phrasing. This is a real risk for a project that selects by accuracy on a conventional split; note, though, that Random Forest is not the worst generalizer overall – under policy-disjoint evaluation it still outperforms six of the seven neural architectures.

4.2 Evaluation-Protocol Gap

Figure 4(b) shows the macro-F1 gap between the two schemes. We call this an *evaluation-protocol gap* rather than a pure measure of leakage because the two schemes also differ in test-set size and policy composition; the 95.2%-versus-6.0% clause-overlap evidence makes leakage the most plausible dominant contributor, but we do not treat every decimal point as causally isolated to leakage alone.

The gap ranges from +0.015 (BERT Base) to +0.123 (Bidirectional SimpleRNN). The three classical models and BERT Base form a comparatively low-gap cluster (+0.015 to +0.070); the six recurrent architectures, trained from scratch with no external pretraining, show both the largest and the most variable gap (+0.041 to +0.123), with Bidirectional SimpleRNN, Bidirectional GRU, and unidirectional LSTM each exceeding +0.10. BERT Base’s gap of only +0.015 is consistent with the hypothesis that pretraining on a large external corpus reduces reliance on this dataset’s own boilerplate, though we did not run the ablation that would isolate pretraining as the cause. SimpleRNN’s small gap should not be read as robustness – its absolute performance is poor under both schemes (0.13–0.17), leaving little headroom for a large gap regardless of leakage sensitivity.

Taken together with the accuracy trap above, the central finding is that a conventional stratified split, scored on accuracy or on an unexamined single-split macro-F1, would have led this project to rank several recurrent architectures well above their honest positions – silently, with nothing in the leaky split’s own numbers to flag the problem.

4.3 Per-Class Analysis

Table 4 reports per-class results for BERT Base under the policy-disjoint test set, and Figure 5 shows the corresponding row-normalized confusion matrix.

Table 4: Per-class P/R/F1 for BERT Base (policy-disjoint; accuracy 0.715, macro-F1 0.706).

Category	P	R	F1
Policy Change	0.91	0.77	0.84
Intl. & Spec. Audiences	0.86	0.88	0.87
Data Security	0.77	0.88	0.82
First Party Coll./Use	0.76	0.78	0.77
Third Party Shar./Coll.	0.66	0.64	0.65
User Access/Edit/Del.	0.69	0.57	0.62
User Choice/Control	0.56	0.62	0.59
Data Retention	0.86	0.35	0.49

Table 3: Test performance at each model’s best validated configuration under both split schemes, ranked by macro-F1. Bold marks the best value in each column within each scheme.

Scheme	Model	Accuracy	Macro-F1	Weighted-F1
random	Logistic Regression	0.7740	0.7534	0.7713
random	Random Forest	0.7798	0.7212	0.7739
random	Naive Bayes	0.7575	0.7258	0.7547
random	BERT Base	0.7586	0.7208	0.7553
random	Bidirectional GRU	0.7640	0.6999	0.7611
random	Bidirectional LSTM	0.7598	0.6840	0.7568
random	Bidirectional SimpleRNN	0.7283	0.6118	0.7222
random	LSTM	0.6914	0.5489	0.6826
random	GRU	0.7110	0.5194	0.7043
random	SimpleRNN	0.4823	0.1719	0.4080
policy	BERT Base	0.7152	0.7060	0.7138
policy	Naive Bayes	0.7026	0.7011	0.6987
policy	Logistic Regression	0.6789	0.6923	0.6770
policy	Random Forest	0.6771	0.6516	0.6673
policy	Bidirectional LSTM	0.6699	0.6083	0.6663
policy	Bidirectional GRU	0.6331	0.5859	0.6344
policy	Bidirectional SimpleRNN	0.6175	0.4891	0.6090
policy	GRU	0.5850	0.4755	0.5832
policy	LSTM	0.5794	0.4425	0.5764
policy	SimpleRNN	0.3863	0.1310	0.3357

Class frequency is a poor predictor of per-class difficulty: *Policy Change* and *International and Specific Audiences* are among the smallest categories yet score highest, because both use distinctive formulaic vocabulary (“we may update this policy from time to time”; “residents of the European Economic Area”) that a model learns from relatively few examples.

The dominant error mode is confusion between *First Party Collection/Use* and *Third Party Sharing/Collection*: both describe data collection and differ chiefly in who receives the data, a distinction sometimes carried by a single noun phrase within a short span. Despite being the two largest categories, they post the two weakest F1 scores among the frequent classes (0.77 and 0.65). *Data Retention* is the one category where scarcity appears to matter: precision is high (0.86) but recall is the lowest of any category (0.35), consistent with a model that recognizes the category when it commits to it but has too few training examples to do so reliably. For every other category, class size and per-class F1 are essentially unrelated ($r \approx 0.05$ across the eight classes).

Best and worst configurations. Among the 60 tuning runs, the winning configuration for BERT Base under the policy-disjoint scheme is $\text{lr} = 3 \times 10^{-5}$, batch size 32 (validation macro-F1 0.723, test macro-F1 0.706). The winning classical configuration is Naive Bayes with $\alpha = 0.05$ (validation macro-F1 0.680, test macro-F1 0.701). The weakest configuration across all models is SimpleRNN with $\text{embed}=128$, $\text{hidden}=64$, $\text{dropout}=0.3$, $\text{lr}=10^{-3}$ under the policy scheme (validation macro-F1 0.110). The full 60-run log is in the project notebook (project_440.ipynb).

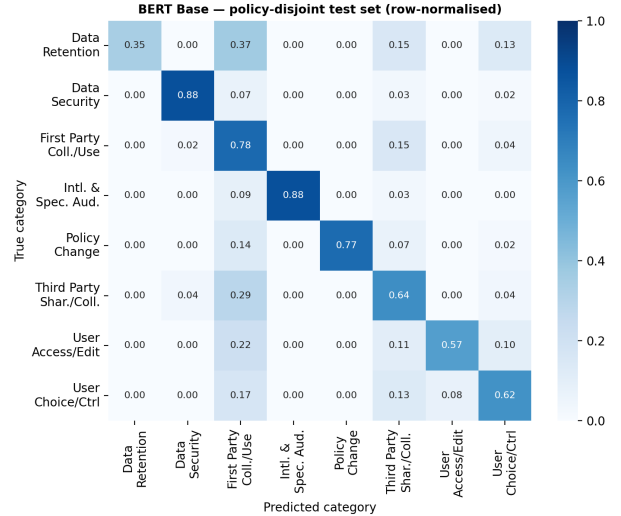


Figure 5: Row-normalised confusion matrix for BERT Base on the policy-disjoint test set. The dominant off-diagonal mass lies between *First Party Collection/Use* and *Third Party Sharing/Collection*.

4.4 Representation Status

TF-IDF is the only representation with completed downstream classification; every classical model result in Table 3 uses TF-IDF. The word2vec embedding model trains successfully on the project vocabulary but no downstream classifier was evaluated on the resulting document vectors. GloVe was not implemented. BERT Base results come from partial, layer-frozen fine-tuning; no full fine-tune was attempted.

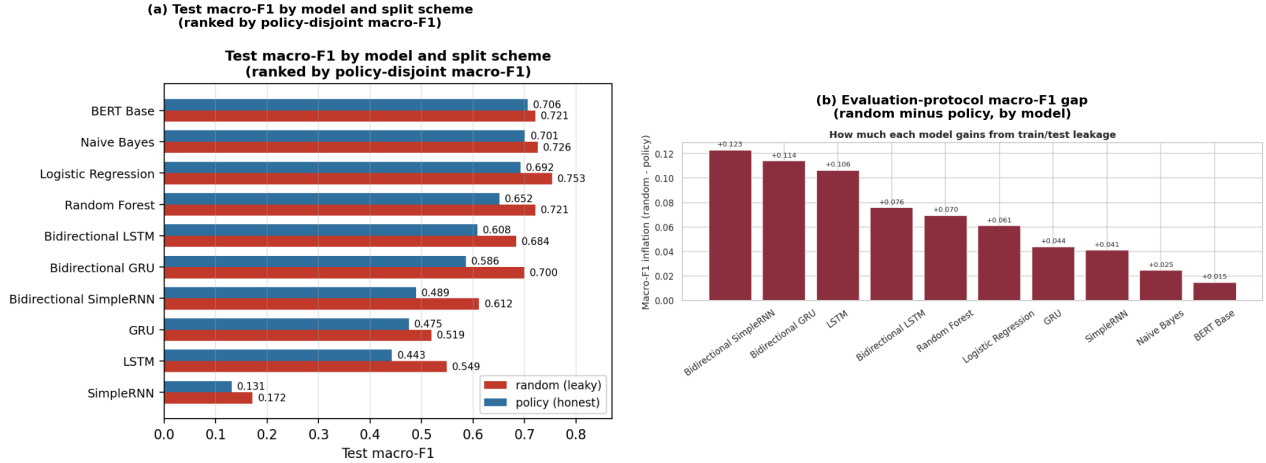


Figure 4: (a) Test macro-F1 for all ten models under both split schemes, ranked by policy-disjoint macro-F1. (b) Evaluation-protocol macro-F1 gap (random-split score minus policy-disjoint score) by model; larger bars indicate models whose apparent performance depends more heavily on signal not available under honest evaluation.

5 Conclusion

Key findings. Evaluating on a conventional row-level split changes not just the absolute numbers but which model appears to win (RQ1). The evaluation-protocol gap is largest for the recurrent architectures trained from scratch and smallest for BERT Base and the classical models (RQ2). Under honest, policy-disjoint evaluation, BERT Base leads at macro-F1 0.706, narrowly ahead of Naive Bayes (0.701); the classical models’ competitiveness suggests that privacy-practice categories carry strong lexical cues that TF-IDF captures effectively on short spans (RQ3). We take the leakage-versus-honest contrast as evidence that document-grouped evaluation should be standard practice for any text classification task built on documents with reused, template-like language – contracts, regulatory filings, terms of service, form letters.

Limitations. The six recurrent models process every padded position in a 48-token sequence without length masking; short spans continue being processed through trailing padding, which may partly explain the recurrent family’s large evaluation-protocol gap. The executed pipeline applied base preprocessing to the recurrent models rather than the designed lemmatized variant; the isolated effect is untested. Random seeds are not reset per configuration inside the recurrent tuning loop. Only TF-IDF downstream classification is complete; word2vec was not evaluated and GloVe was not implemented. BERT results come from partial, layer-frozen fine-tuning rather than a full fine-tune. All results are oracle-span: a deployed system would additionally need an upstream span-detection step.

Future work. The most pressing fixes are retraining the six recurrent models with length-aware packing and per-configuration seed resets, completing the word2vec downstream evaluation, and applying the designed lemmatized pre-

processing to the recurrent models to isolate its effect. Beyond that, the 34.9% of clauses that carry multiple categories suggest feeding span-plus-clause context should specifically help the *First Party* versus *Third Party* confusion, since the distinguishing noun phrase often sits just outside the annotated span. Minority-class-aware training strategies should help *Data Retention* – the one category where scarcity is the bottleneck – more than the semantically overlapping majority categories. A full, non-frozen BERT fine-tune given GPU access is expected to leave real headroom above the reported figures.

References

- Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O’Reilly Media.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of EMNLP*, 1724–1734. <https://doi.org/10.3115/v1/D14-1179>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Gorman, K., & Bedrick, S. (2019). We need to talk about standard splits. *Proceedings of ACL*, 2786–2791. <https://doi.org/10.18653/v1/P19-1267>
- Harkous, H., Fawaz, K., Lebre, R., Schaub, F., Shin, K. G., & Aberer, K. (2018). Polis: Automated analysis and presentation of privacy policies using deep learning. *Proceedings of USENIX Security*, 531–548.

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Johnson, J. M., & Khoshgoftaar, T. M. (2019). Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1), Article 27. <https://doi.org/10.1186/s40537-019-0192-5>
- Lewis, P., Stenetorp, P., & Riedel, S. (2021). Question and answer test-train overlap in open-domain question answering datasets. *Proceedings of EACL*, 1000–1008.
- McDonald, A. M., & Cranor, L. F. (2008). The cost of reading privacy policies. *I/S: A Journal of Law and Policy for the Information Society*, 4(3), 543–568.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *ICLR Workshop*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*, 32, 8024–8035.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Pennington, J., Socher, R., & Manning, C. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP*, 1532–1543. <https://doi.org/10.3115/v1/D14-1162>
- Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC Workshop on New Challenges for NLP Frameworks*, 45–50.
- Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681. <https://doi.org/10.1109/78.650093>
- Wilson, S., Schaub, F., Dara, A. A., Liu, F., Cherivirala, S., Leon, P. G., Andersen, M. S., Zimmeck, S., Sathyendra, K. M., Russell, N. C., Norton, T. B., Hovy, E., Reidenberg, J., & Sadeh, N. (2016). The creation and analysis of a website privacy policy corpus. *Proceedings of ACL*, 1330–1340. <https://doi.org/10.18653/v1/P16-1126>
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger,
- S., Drame, M., Lhoest, Q., & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of EMNLP: System Demonstrations*, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>